

Vue 3 Dasar Terpadu

Lifecycle, Logic Reusable, State, dan Styling

Component



Konteks Pembelajaran

Materi Sebelumnya

Anda telah menguasai Setup Vite, Sintaks Template, Komponen Dasar, Props, Reaktivitas Dasar, Event Handling, serta Conditional & List Rendering.

Fokus Modul Ini

Kita akan memperdalam struktur kode Vue dengan Lifecycle, akses DOM dengan Ref, State Management (Computed, Watch, Provide), dan pembuatan Custom Composable yang modular.

Tujuan Pembelajaran



Lifecycle Hooks: Menangani logika pada fase kelahiran dan kematian komponen.



Template Refs: Mengakses elemen DOM secara aman dari script.



Computed & Watch: Mengelola nilai turunan reaktif dan memantau efek samping.



Provide & Inject: Mendistribusikan data ke seluruh hierarki komponen.



Custom Composable: Membuat fungsi reaktif yang mudah digunakan kembali.

Alur Materi

1. Lifecycle

Memahami fase komponen dari awal dirender hingga dicabut.

2. State Logic

Mendalami Computed Properties dan Watcher.

3. Modularitas

Menyelami teknik Provide/Inject dan Custom Composable.

Pergeseran Mental Model



Pola Lama

State berinteraksi langsung ke Template untuk menjadi UI. Pola ini sangat kaku dan hanya efektif pada aplikasi sederhana.



Pola Vue Modern

State diproses oleh Computed/Watch, bereaksi melalui fase Lifecycle, disusun dalam Composable modular, dan dieksekusi di DOM.



Lifecycle Hooks

Mengatur eksekusi kode berdasarkan fase siklus hidup komponen

Apa Itu Lifecycle?

Siklus

Lahir hingga Mati

Perjalanan Komponen

Setiap komponen Vue melewati fase otomatis saat ia bekerja: Dibuat di memori, disuntikkan ke DOM (Mounted), diperbarui karena state berubah (Updated), hingga dicabut atau dihapus dari layar (Unmounted).

Vue menyediakan jendela (Hooks) untuk kita menempelkan kode spesifik di momen yang sangat persis tersebut.

Mengapa Lifecycle Dibutuhkan?



Mengambil Data (Fetch): Menarik data dari API tepat setelah komponen selesai digambar.



Akses DOM Eksternal: Melakukan inisialisasi library pihak ketiga (seperti Chart.js) yang membutuhkan elemen HTML fisik.



Pembersihan Memori: Membuang timer (setInterval) atau event listener sebelum komponen dihancurkan agar aplikasi tidak lambat.

Sintaks Hooks di Composition API

Dalam metode Composition API, seluruh Lifecycle Hooks adalah fungsi yang harus diimpor langsung secara manual dari library Vue.

Hooks ini selalu dipanggil di tingkat paling atas dari bagian `<script setup>`.

```
<script setup>
import {
  onBeforeMount,
  onMounted,
  onBeforeUpdate,
  onUpdated,
  onBeforeUnmount,
  onUnmounted
} from "vue"
</script>
```

Tiga Hook Terpenting



onMounted

Dijalankan pertama kali setelah struktur HTML template selesai digambar sempurna di browser.



onUpdated

Dieksekusi ulang setiap kali tampilan berubah atau render ulang merespons pergantian nilai state.



onUnmounted

Fase pemusnahan, dipanggil persis sesaat sebelum elemen dicabut secara permanen dari layar.

Penggunaan onMounted

Target Eksekusi

Hook ini sangat sempurna untuk pekerjaan yang mutlak membutuhkan ketersediaan DOM.

Contohnya membaca dimensi elemen atau melakukan fetch data awal ke server.

Contoh Kode

```
onMounted(() => {  
  console.log("DOM Siap!")  
  // Mulai panggil API di sini  
  fetchData()  
})
```

Siklus Pembaruan: onUpdated

Setiap ada mutasi reaktif pada variabel `ref()`, Vue menjadwalkan pembaharuan UI. Begitu UI selesai diperbarui, `onUpdated()` akan memanggil aksi lanjutan.

Gunakan hook ini dengan hati-hati. Hindari merubah ulang state di dalam hook ini karena berpotensi menciptakan infinite loop (loop tanpa henti).

```
<script setup>
import { ref, onUpdated } from "vue"

const count = ref(0)

onUpdated(() => {
  console.log("UI terupdate:", count.value)
})
</script>
```

Krusialnya Cleanup (onUnmounted)



Memory Leak: Ketika komponen dihapus (misal berpindah halaman rute), proses di latar belakang seperti setInterval tidak otomatis mati.



Tugas onUnmounted: Anda wajib membuang timer, memutuskan koneksi WebSocket, dan menghapus Event Listener manual di fase ini.



Prinsip Emas: Apapun yang Anda "create" di onMounted, wajib pasangkan dengan instruksi "destroy" di onUnmounted.

Urutan Lengkap Hook Vue

Hook "Before" (Persiapan)	Hook "Selesai" (Finalisasi)
onBeforeMount: Segera dirender	onMounted: Telah tercetak di layar
onBeforeUpdate: Data baru masuk	onUpdated: Layar selesai mencetak data
onBeforeUnmount: Persiapan cabut	onUnmounted: Lenyap dari aplikasi



Template Refs

Akses langsung ke elemen HTML secara spesifik

Apa Itu Template Refs?

Keluar Dari Konteks Reaktif

Vue merender komponen berdasar state (Deklaratif). Namun terkadang kita harus memberi perintah mutlak langsung ke suatu elemen (Imperatif).

Template Refs adalah cara Vue memberikan kita izin menyentuh elemen DOM nyata secara langsung menggunakan ref.



Membuat dan Mengakses Refs

Langkah Pengikatan

Buat variabel null menggunakan `ref()`. Lalu lekatkan nama variabel tersebut pada tag HTML menggunakan atribut bawaan `ref="namaVariabel"`.

Sintaks Kode

```
<script setup>
import { ref, onMounted } from "vue"
const inputEl = ref(null)

onMounted(() => {
  inputEl.value.focus() // Akses DOM
})
</script>
<template>
  <input ref="inputEl">
</template>
```

Kasus Penggunaan Template Ref



Autofocus Input

Memaksa kursor aktif pada isian tertentu tepat saat halaman selesai dimuat tanpa harus diklik oleh pengguna.



Membaca Dimensi

Mendapatkan informasi ukuran pasti (`offsetWidth`, `offsetHeight`) atau jarak scroll dari elemen khusus di layar.



Library Eksternal

Menyerahkan kontrol suatu div kosong ke plugin JavaScript di luar ekosistem (contoh: Google Maps API).

Referensi ke Komponen Anak (Child)

Selain digunakan pada tag HTML biasa, ref juga dapat diletakkan pada tag Child Component. Ini memberikan Parent akses ke fungsi publik milik si Anak.

```
<ChildComponent ref="childRef" />
```

Namun secara default, script setup Vue bersifat sangat privat. Anda harus mengizinkannya terlebih dahulu dari sisi anak.

```
/* Pada Komponen Anak */  
<script setup>  
function sayHello() {  
  alert("Halo dari Anak!")  
}  
  
defineExpose({  
  sayHello  
})  
</script>
```



Computed Properties

Mengelola nilai turunan dari state reaktif yang terkomputasi otomatis

Apa Itu Computed?

Auto

Turunan State

Meracik Nilai Baru

Computed adalah fungsi yang menghasilkan turunan state reaktif (derived state). Jika sebuah fungsi hanya bertugas menghitung dan mengembalikan gabungan data reaktif yang ada, maka itu adalah Computed.

Contoh klasik: Menggabungkan firstName dan lastName menjadi sebuah properti fullName yang mulus.

Sintaks Computed

Deklarasi Fungsi

Fungsi `computed` menerima parameter berupa fungsi callback yang wajib mengembalikan (return) nilai akhir.

Contoh Kode

```
import { ref, computed } from "vue"

const harga = ref(100)
const jumlah = ref(2)

const total = computed(() => {
  return harga.value * jumlah.value
})
```

Karakteristik Utama Computed



Otomatis Reaktif: Bila variabel harga atau jumlah berubah, maka total akan langsung diperbarui tanpa intervensi.



Sistem Caching: Berbeda dari metode (function) biasa, Computed menyimpan hasil hitungannya. Ia tidak akan menghitung ulang jika nilai dasarnya tidak mengalami perubahan sedikitpun.



Hanya Untuk Turunan: Jangan pernah merubah DOM atau memanggil API Server dari dalam computed.



Watchers

Pengintai state reaktif untuk mengeksekusi operasi lanjutan

Memahami Watchers

Side Effect Eksekutor

Watcher mengintai spesifik pada suatu state. Bila state tersebut berubah, watcher tidak menghasilkan nilai baru (seperti computed), melainkan memicu efek samping (Side Effect) atau aksi.

Pekerjaan berat yang tidak boleh dilakukan Computed, justru adalah tugas dari Watcher.



Aturan Emas: Computed vs Watch

Parameter Penilaian	Gunakan Computed	Gunakan Watcher
Tujuan Utama	Menghasilkan nilai / data turunan baru.	Melakukan pekerjaan saat data berubah.
Contoh Implementasi	Total harga, Filter Pencarian Array.	Kirim HTTP Request (Fetch), Notifikasi, Log.
Aturan Logika	Wajib ada Return, Dilarang Side Effect.	Bebas operasi asinkronous berat.

Sintaks Dasar Watch

Target & Callback

Fungsi `watch` mengambil dua parameter minimal: (1) state mana yang diawasi, dan (2) fungsi callback yang membawa `newValue` dan `oldValue`.

Kode Implementasi

```
import { ref, watch } from "vue"
const count = ref(0)

watch(count, (newVal, oldVal) => {
  console.log(`Berubah: ${oldVal} → ${newVal}`)
  // Jalankan Fetch API ...
})
```

Opsi Spesial: Immediate & Deep



Immediate

Secara default watch hanya berjalan ketika nilai berubah. Tambahkan `immediate: true` jika ingin watcher langsung dijalankan pada detik pertama komponen dimuat.



Deep

Untuk mengawasi seluruh kedalaman object atau array bersarang yang rumit, sisipkan konfigurasi `deep: true` agar sistem memantaunya lapis demi lapis.



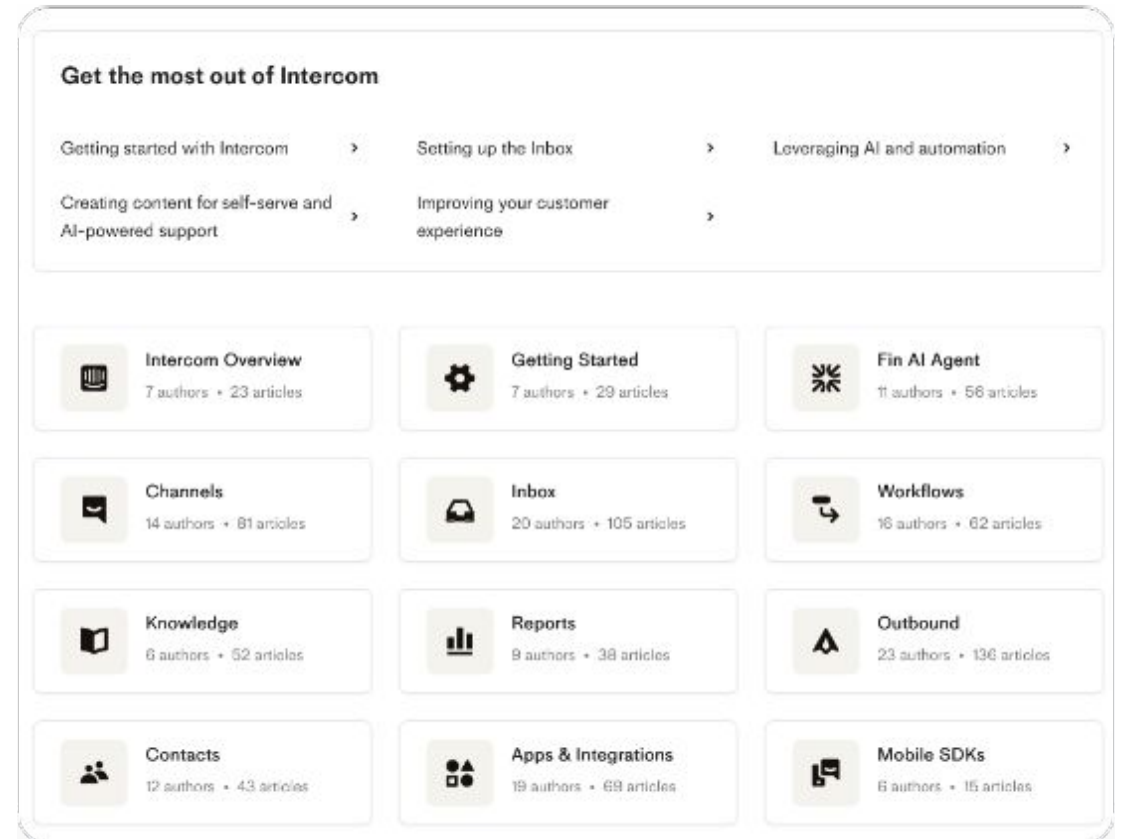
Provide & Inject

Solusi ampuh menghindari mimpi buruk transfer Props berantai

Masalah: Props Drilling

Transfer yang Melelahkan

Jika Komponen Level 1 ingin mengirim data ke Komponen Level 5, data tersebut harus dilempar satu per satu sebagai Props menembus Level 2, 3, dan 4, padahal komponen tengah tidak membutuhkannya sama sekali.



Pola Provide & Inject

Provide (Penyedia)

Komponen Induk Utama (Ancestor) mendaftarkan sebuah kunci dan data. Ia meletakkannya di ruangan tunggu virtual yang bisa diakses seluruh keturunannya.

```
import { provide } from "vue"
provide("theme", "dark")
```

Inject (Pengambil)

Komponen keturunan level berapapun bebas "menyedot" data tersebut dari ruangan virtual cukup dengan berbekal kunci penandanya.

```
import { inject } from "vue"
const theme = inject("theme")
```

Kasus Penggunaan Terbaik



Tema Tampilan: Mendistribusikan konfigurasi tema (Light/Dark mode) ke semua elemen tombol dan teks di dalam aplikasi.



Data Autentikasi: Menyebarkan objek status user aktif yang sudah terverifikasi tanpa membebani hirarki props komponen.



Peringatan Penting: Ini bukan pengganti total manajemen state (seperti Pinia). Gunakan hanya untuk ketergantungan konseptual statis secara struktural.



Custom Composable

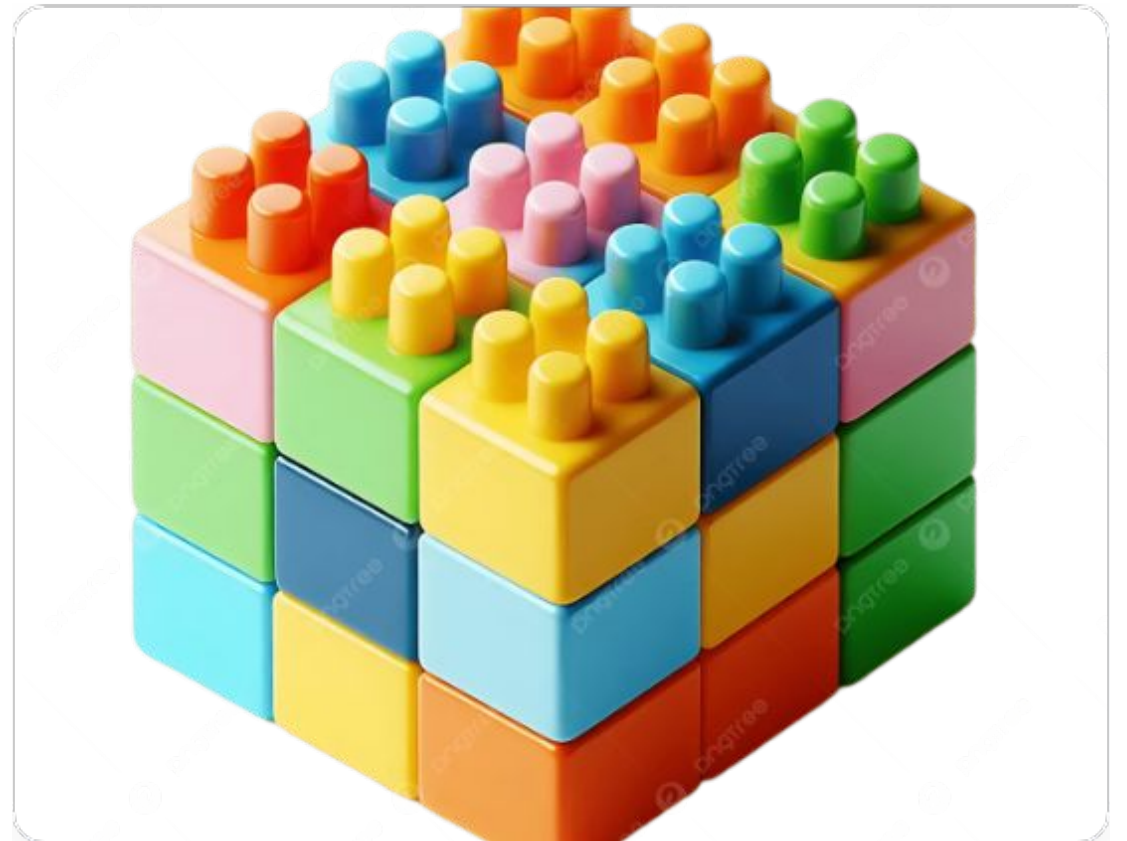
Membentuk modul fungsi logika yang dapat digunakan berulang
(Reusable Logic)

Konsep Custom Composable

Kapsul Logika Reaktif

Berkat Vue 3 Composition API, kita bebas membungkus logika state, fungsi computed, hingga pemanggilan lifecycle ke dalam sebuah file JavaScript murni independen.

Kumpulan kode ini disebut **Composable**, yang bebas Anda impor dan eksekusi ulang di puluhan komponen layar yang berbeda tanpa resiko tabrakan variabel.



Struktur File Composable

Aturan Penamaan

Secara konvensi dan tata krama standar, nama file atau nama fungsi wajib selalu diawali dengan suku kata awalan use... (contoh: useFetch, useCounter).

Ekspor Objek

Di penghujung kode fungsinya, wajib lakukan return mencakup variabel state dan fungsi metodologinya agar komponen luar bisa menarik datanya utuh.

Implementasi: useCounter.js

Memisahkan kode reaktif agar lebih modular dan file komponen Vue Anda menjadi sangat bersih serta mudah dibaca fokus pada User Interface belaka.

```
import { ref } from "vue"

export function useCounter() {
  const count = ref(0)
  const increment = () => count.value++

  return { count, increment }
}
```

Panggilan di dalam Komponen App.vue:

```
<script setup>
import { useCounter }
  from "../composables/useCounter"

const { count, increment } = useCounter()
</script>
```



Styling di Vue

Pendekatan mewarnai dan mendesain elemen antarmuka

Metode Penerapan Desain



Inline Style: Menggunakan `:style="{ color: 'red' }"`. Cocok untuk nilai dinamis mutlak yang spesifik dari JavaScript.



Class Binding: `:class="{ active: isActive }"`. Standar emas mengaktifkan ragam variasi desain berbasis kondisi state.



Scoped Style: Melampirkan atribut khusus `</div>` memastikan modifikasi desain Anda tidak akan merembes menghancurkan komponen tetangga.



Global Style: Impor file CSS standar (main.css) untuk reset dasar margin dan hierarki tipografi sistem.

Desain Berbasis Reaktivitas

Sintaks Object pada Class

Manfaatkan perantara objek, jika nilai boolean-nya true, Vue akan merender nama kelas tersebut utuh ke dalam pohon DOM.

```
<p :class="{ active: isOk }">  
  Teks Interaktif  
</p>
```

Style Dinamis Menyeluruh

Variabel referensi lebar, warna yang dipilih user, bisa disisipkan seketika merubah gaya desain secara radikal dengan sangat mulus.

```
<div :style="{ width: lebar + 'px' }">
```

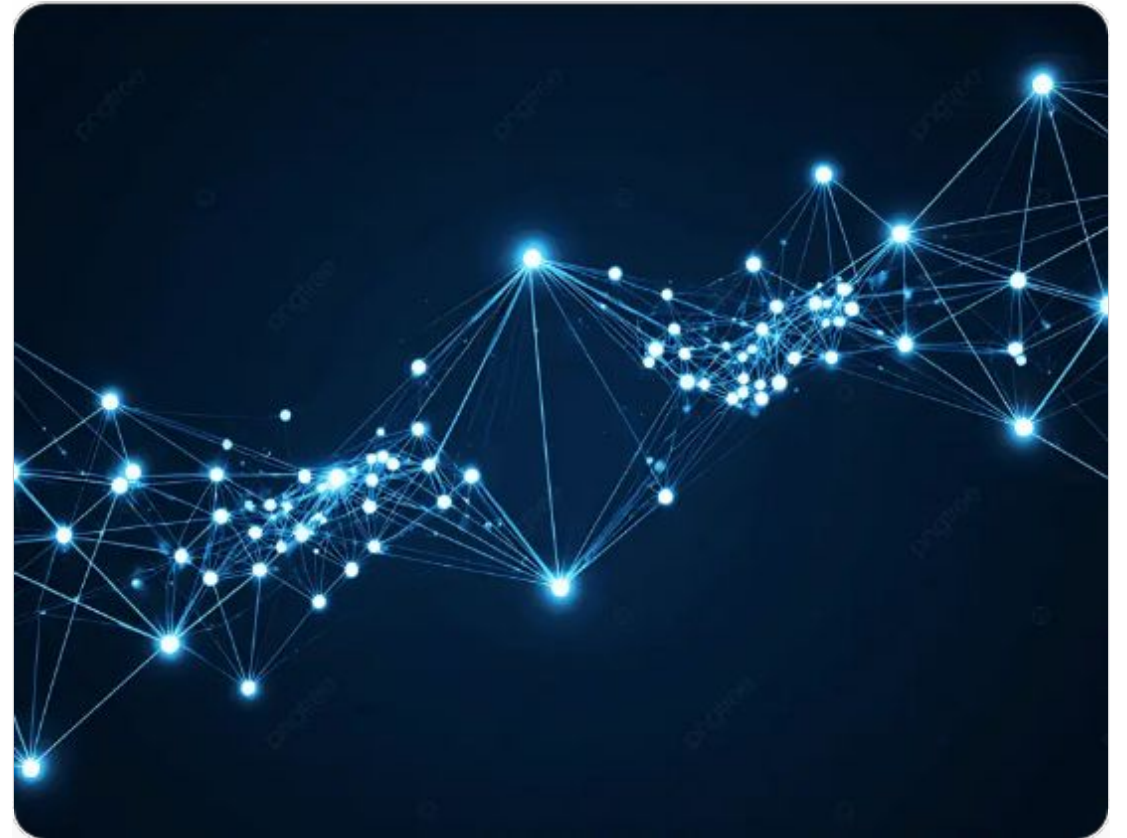
Perbandingan Lingkup Gaya (CSS)

Parameter Penilaian	(Lokal)	Global CSS (main.css)
Area Pengaruh	Eksklusif hanya di dalam file komponen terkait.	Mempengaruhi seluruh penjuru layar aplikasi.
Pencegahan Konflik	Sangat aman, nama class .btn takkan berbenturan.	Risikannya bertabrakan jika tatanama tidak diatur ketat.
Penggunaan Terbaik	Desain Kartu Profil, Tombol Aksi khusus, Modals.	Font dasar, Reset Margin, Tema Warna Utama.

Integrasi Seluruh Fitur Bersamaan

Membangun Arsitektur

Aplikasi web modern (contoh: Dashboard) akan menggabungkan semuanya. **Composable** meracik operasi Fetch (Watcher) dan Data Filter (Computed). State diturunkan dengan **Provide**, lalu UI dirender pada saat fase **Mounted** dengan pembatas desain **Scoped Styling**.



Kesalahan Fatal Umum – Part 1



Akses DOM Prematur: Mengeksekusi `input.focus()` sebelum masuknya fase krusial `onMounted`, sehingga elemen mutlak belum diciptakan ke layar peramban.



Kelupaan Pembersihan (Cleanup): Menyebarkan jerat timer interval dan pemicu soket terbuka tanpa ditutup di `onUnmounted` yang menyebabkan browser macet berat.

Kesalahan Fatal Umum – Part 2



Salah Penggunaan Watcher: Memforsir pemakaian `watch()` sekadar untuk merumuskan ulang penggabungan data variabel sederhana alih-alih mengandalkan tenaga instan `computed()`.



Gagal Props Drilling: Melemparkan data berantai melintasi hingga 5 turunan anak komponen secara manual yang memusingkan, padahal kunci `Provide/Inject` sudah disiapkan.

Mental Model Pengembangan Vue Elite

Pertanyaan Arsitektural Utama

1. Kapan blok logika dieksekusi? (**Lifecycle**)
2. Butuh pegang elemen keras? (**Ref**)
3. Hanya hasil turunan operasi? (**Computed**)

Alur Komunikasi Aplikasi

4. Adakah respon efek lanjutan? (**Watch**)
5. Harus dikirim melompati hirarki? (**Provide**)
6. Pola kodenya bisa dicetak ulang? (**Composable**)

Ada pertanyaan?

Anda telah menelusuri alur mendalam Vue 3 mulai dari titik nol komponen, efek reaktif kompleks, sampai pemisahan modul mutakhir.